

听言英语 V1.0 程序设计说明书

软件名称：听言英语（Echo Ling）
软件版本：V1.0
开发平台：Android（原生）
包名：com.echoling.app
文档定位：用于计算机软件著作权登记的程序设计说明文档

目录

- 1. 引言 3
 - 1.1 编写目的 3
 - 1.2 编写依据 3
 - 1.3 术语与缩略语 3
- 2. 项目概述 3
 - 2.1 软件简介 3
 - 2.2 核心功能 4
 - 2.3 软件特点 4
- 3. 开发与运行环境 4
 - 3.1 硬件环境（开发机） 4
 - 3.2 软件环境（开发端） 5
 - 3.3 软件环境（运行端） 5
 - 3.4 主要第三方依赖 5
 - 3.5 资源与数据文件 5
- 4. 需求分析 6
 - 4.1 功能性需求 6
 - 4.2 非功能性需求 7
 - 4.3 权限需求 7
- 5. 系统架构设计 8
 - 5.1 总体架构 8
 - 5.2 模块划分 9
 - 5.3 依赖注入（Hilt） 9
 - 5.4 协程与并发 9
 - 5.5 导航架构 9
- 6. 模块详细设计 10
 - 6.1 应用入口模块 10
 - 6.2 视图模型层（ViewModel） 11
 - 6.3 域模型层 11
 - 6.4 数据层 13
 - 6.5 播放引擎模块（Player） 14
 - 6.6 语音处理模块（Speech） 14
 - 6.7 UI 层 15

7. 数据库设计	16
7.1 数据库概述	16
7.2 ER 图（文字描述）	16
7.3 表结构	17
7.4 DAO 设计	18
7.5 版本历史	19
8. 关键算法设计	19
8.1 跟读测试端到端流程	19
8.2 WordMatcher 词序匹配算法	20
8.3 SRT 字幕解析算法	22
8.4 闪卡 Y 轴 3D 翻转动画	22
8.5 11 词库的嵌套 / 扁平 schema 自动嗅探	23
9. 界面与交互设计	24
9.1 设计语言	24
9.2 关键交互	24
9.3 状态栏 / 导航栏适配	24
10. 性能与可靠性设计	24
10.1 启动性能	24
10.2 内存管理	25
10.3 16 KB 页大小对齐（Android 15+）	25
10.4 隐私设计	25
10.5 错误处理与降级	25
10.6 并发与生命周期	26
11. 测试与验证	26
11.1 单元测试	26
11.2 集成验证（真机实测，小米 Mi 11 CN）	26
11.3 构建验证	26
11.4 兼容性验证	27
12. 人员分工与开发周期	27
12.1 人员构成	27
12.2 开发周期（按工时统计）	27
12.3 关键里程碑	28
13. 附录	28
13.1 关键文件清单	28
13.2 资源清单	29
13.3 构建脚本	30
13.4 文档清单	30
文档结束	30

1. 引言

1.1 编写目的

本说明书面向计算机软件著作权登记审查及后续维护人员，系统描述「听言英语 V1.0」Android 移动应用的整体设计方案、模块划分、关键算法、数据结构与人员分工。审查人员可通过本说明书快速了解软件的技术实现细节，维护人员可据此理解代码组织、模块边界与扩展方式。

1.2 编写依据

- 《计算机软件著作权登记办法》
- 《计算机软件著作权登记文档编写规范》
- 软件对应的源代码包 `app/src/main/java/com/echoling/app` (113 个 Kotlin 源文件)
- 配套源程序文档 `听言英语\_V1.0\_源程序\_全本.pdf` 及 `听言英语\_V1.0\_源程序\_前 30 页\_后 30 页.pdf`

1.3 术语与缩略语

术语	含义
STT	Speech-to-Text, 语音识别
TTS	Text-to-Speech, 文字转语音
MVVM	Model-View-ViewModel 架构模式
UseCase	用例层, 对单一业务逻辑的封装
Repository	仓储层, 统一的数据出入口
Hilt	基于 Dagger 的 Android 依赖注入框架
Room	Android SQLite 对象映射持久化库
Compose	Jetpack Compose, 声明式 UI 框架
ExoPlayer	AndroidX Media3 提供的媒体播放引擎
Vosk	开源离线语音识别引擎
SRT/ASS/LRC	三种常见字幕文件格式
PCMAudio	16 kHz 单声道 16 位 PCM 音频

2. 项目概述

2.1 软件简介

「听言英语」是一款基于 Android 原生平台开发的英语听说训练应用。软件聚焦"通过视频/音频素材进行逐句精听、跟读、复述测试"这一核心训练闭环，配合本地化词典查询、闪卡

式单词记忆等辅助功能，为英语学习者提供一套离线优先、零网络依赖的便携学习工具。

2.2 核心功能

功能模块	简要说明
课程导入	通过系统 SAF (Storage Access Framework) 选择本地音视频素材，自动识别同名字幕文件
字幕解析	支持 SRT / ASS / LRC 三种主流字幕格式的解析与同步
跟读练习	泛听 (自由播放) / 精听 (单句循环) / 测试 (跟读对比) 三段式练习
跟读对比测试	用户按住录音，系统在本地将语音转写为文字后与原字幕逐词匹配对比 (详见 § 8.2)
长按取词	长按字幕单词调出本地词典查询，支持音标、词性、翻译、例句显示，并可一键收藏至生词本
闪卡记单词	11 大词库 (小学 / 初中 / 高中 / CET-4 / CET-6 / CET-8 / 考研 / 托福 / GRE / IELTS / BEC) 的卡片式记忆
单词发音	基于系统 TTS 引擎朗读单词 (多引擎自动 fallback)
学习统计	按天统计学习时长、已学句子数、新增单词数；按月热力图可视化活跃度
权限使用说明	在 App 内集中声明权限用途与隐私承诺

2.3 软件特点

1. ****100% 离线可用****: Vosk 离线识别模型 (约 68 MB) 已内置在 APK 内，运行时不再产生任何网络请求；词典数据全部打包在 APK 内。
2. ****全 Kotlin 原生开发****: 基于 Jetpack Compose + MVVM + Clean Architecture，不引入任何 Java 代码，不使用任何 XML 布局。
3. ****跟读对比测试设计****: 本软件的"复述/跟读测试"采用"语音识别 → 文本 → 与原文逐词对比"的方式实现 (详见 § 8.2)，不引入任何打分、评分或发音质量评估机制。
4. ****数据本地化****: 所有用户学习数据存储于本地 Room 数据库，不上传任何用户数据到云端。

3. 开发与运行环境

3.1 硬件环境 (开发机)

项目	配置
CPU	Intel / Apple Silicon 多核处理器
内存	≥ 16 GB
磁盘	≥ 50 GB 可用空间 (Android SDK + Gradle + 项目源码 + 构建产物)
操作系统	Windows 11 (开发) / macOS / Linux (兼容)

3.2 软件环境（开发端）

项目	版本/说明
JDK	OpenJDK 17 (AGP 8.x 要求)
Android Gradle Plugin	8.5.x
Kotlin	2.0.x
Android SDK	Platform 34 (compileSdk / targetSdk), minSdk 26 (Android 8.0)
Build Tools	34.0.0+
IDE	Android Studio (最新稳定版)
Gradle	Kotlin DSL 构建脚本

3.3 软件环境（运行端）

项目	版本/说明
Android	8.0 (API 26) ~ 14+ (API 34+)
架构	arm64-v8a / armeabi-v7a / x86_64
屏幕	任意分辨率 (适配手机 / 折叠屏)
必要运行时	TTS 引擎 (系统自带或第三方, 如 Google TTS、小爱语音、讯飞语音+)

3.4 主要第三方依赖

库	版本范围	用途
Jetpack Compose BOM	2024.x	声明式 UI
AndroidX Media3 (ExoPlayer)	1.3.x	音视频播放
AndroidX Room	2.6.x	本地数据库
Hilt	2.50.x	依赖注入
Kotlin Coroutines + Flow	1.7.x	异步与响应式流
Vosk Android	0.3.45	离线 STT
OkHttp	4.12.x	(保留依赖, 目前仅用于可选的远端诊断/日志上报, 未实际使用)
AndroidX Navigation Compose	2.7.x	子页面路由
Material3	1.1.2	Material You 设计语言
Gson	2.10.x	词典 JSON 解析

3.5 资源与数据文件

- **11 个词库 JSON 文件** : `vocab\_primary.json` (32 KB) / `vocab\_junior.json` (668 KB) /
`vocab\_senior.json` (844 KB) / `vocab\_cet4.json` (1.8 MB) / `vocab\_cet6.json` (188 KB) /
`vocab\_cet8.json` (1.6 MB) / `vocab\_kaoyan.json` (1.2 MB) / `vocab\_toefl.json` (1.4 MB) /

`vocab\_ielts.json` (1.2 MB) / `vocab\_gre.json` (2.0 MB) / `vocab\_bec.json` (952 KB)
- ****1 个 manifest****: `vocab\_manifest.json` (4 KB) 描述 11 个分类的元数据
- ****Vosk 离线模型****: 约 68 MB, 已****内置****在 APK `assets/models/` 目录中, 首次启动时由系统自动解压部署至内部存储 (无需任何网络下载)
- ****应用图标 / 启动图****: `ic\_app\_icon.png` / `splash\_image.png` / 各种矢量图标

4. 需求分析

4.1 功能性需求

4.1.1 课程管理

- 用户可导入本地的 mp4 / mkv / mp3 / wav 文件作为练习素材
- 系统自动在同目录下查找与媒体文件同名的 `.srt` / `.ass` / `.lrc` 字幕文件
- 若未找到同名字幕, 提示用户手动选择字幕文件
- 导入完成后, 课程卡片出现在"首页"列表中

4.1.2 跟读练习

- 课程详情页展示课程元信息 (标题、描述、难度、时长、句子数)
- 进入跟读练习后, 提供三个 Tab:

Tab	名称	主要操作
泛听	Listening	自由播放; 点击字幕条目跳到对应句播放
精听	Speaking	单句循环; 可点单词隐藏/显示; 可录音并回放
测试	Testing	按住麦克风录音 → 转写文字 → 与原字幕逐词对比

4.1.3 跟读测试核心规则

- > ****重要设计声明****:
- > 本软件的跟读测试采用"语音识别 → 文本 → 与原字幕逐词对比"的方式实现。
 - > 系统将用户说出的话转写为英文文本后, ****通过词序匹配**** (详见 § 8.2 WordMatcher 算法) 将识别结果与原字幕逐词对比, 并在 UI 上以"绿底=匹配 / 红底=未匹配"的颜色块直观反馈每个单词的命中情况。
 - > 本系统****不包含任何打分、评分、发音质量评估机制****——测试结果仅为"通过 / 未通过"二元判定, 外加按词展示的命中详情, 不存在"本次复述得分 X 分"之类的评分概念。

4.1.4 词典查询与生词本

- 用户在泛听 / 精听页面长按字幕单词，弹出"翻译 & 保存"对话框
- 对话框显示单词、音标、词性、翻译、例句（若有）
- 用户可一键保存至生词本，生词本独立持久化于 Room 表 `vocabulary`

4.1.5 闪卡记单词

- 11 大词库以分类卡片形式展示在"记单词"主页
- 进入分类后展示闪卡（正方形卡片，点击触发 Y 轴 3D 翻转查看翻译）
- 用户点击"认识"或"不认识"后自动翻到下一张；进度自动持久化

4.2 非功能性需求

- | 需求类别 | 具体要求 |
- | 启动性能 | 冷启动 $\leq 1.5\text{ s}$ （小米 Mi 11 实测） |
- | 离线能力 | 除模型首次部署外，所有功能均无需网络 |
- | 隐私 | 不请求 INTERNET 权限；不收集任何用户数据；权限使用说明在 App 内明示 |
- | 屏幕适配 | 支持 Android 8.0 ~ 14+；支持深色 / 浅色主题 |
- | 多窗口 | 支持折叠屏 / 平板的多窗口模式 |
- | 16 KB 对齐 | 已对齐 ELF PT_LOAD.p_align、.so zip entry、resources.arsc，兼容 Android 15+ 16 KB 页大小设备 |
- | 本地化 | 中文界面（strings.xml 国际化准备中） |
- | 编译产物 | Debug APK $\approx 30\text{ MB}$ （含 11 个词库 JSON + Vosk 模型）；Release APK 经 R8 缩减 |

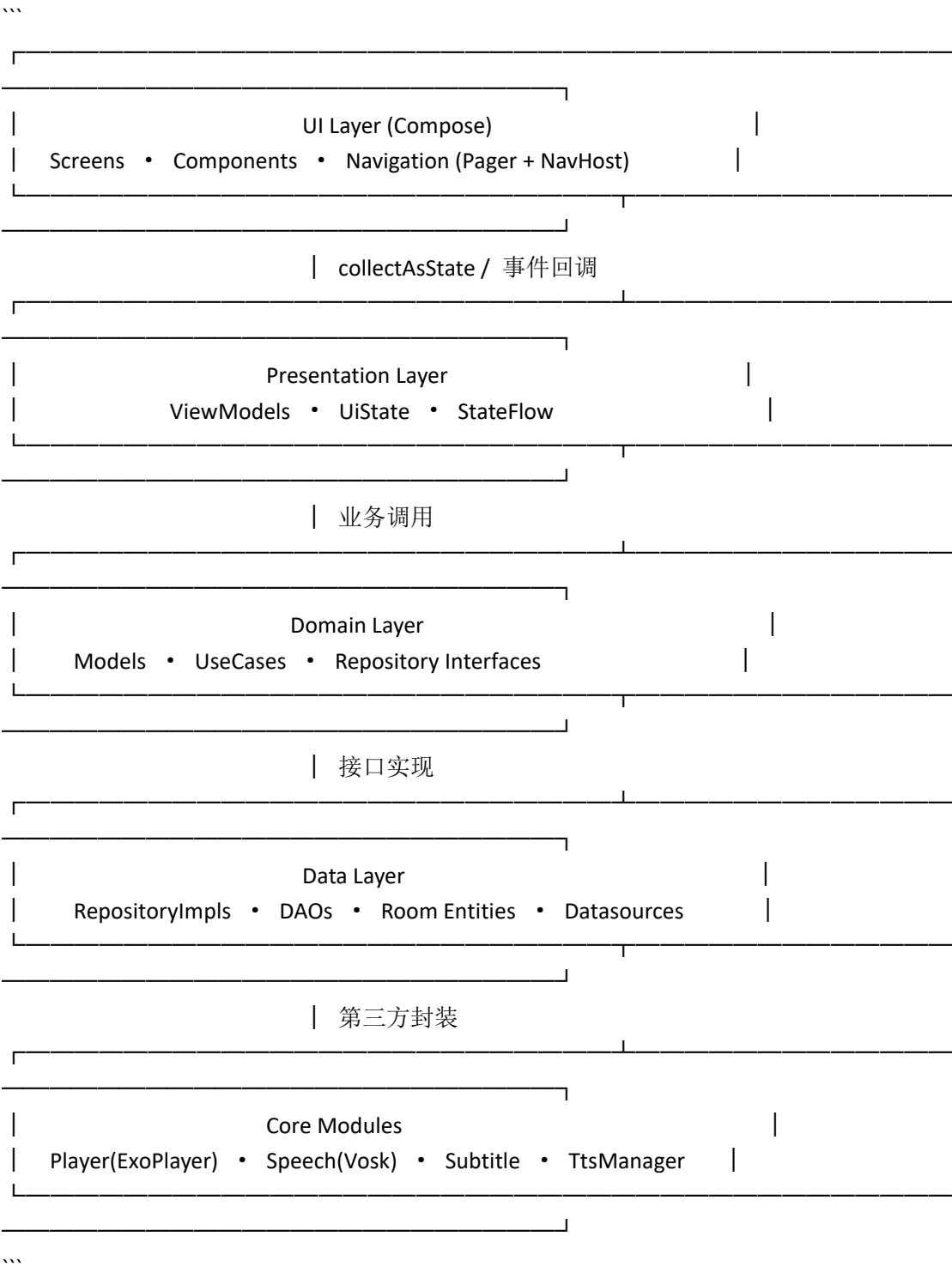
4.3 权限需求

- | 权限 | 用途 | 申请时机 |
- | `RECORD\_AUDIO` | 跟读测试 / 精听录音 | 用户首次按下麦克风按钮时申请 |
- | `READ\_MEDIA\_VIDEO` / `READ\_MEDIA\_AUDIO` | 通过 SAF 选择媒体文件 | 不直接申请，使用 SAF 系统选择器 |
- | `QUERY\_ALL\_PACKAGES` | 检测已安装的 TTS 引擎 | 通过 `` 声明 |
- | `INTERNET` | **不使用**，已刻意从 manifest 移除（详见 § 10.4 隐私设计） | —— |

5. 系统架构设计

5.1 总体架构

软件采用 **Clean Architecture（整洁架构）+ MVVM** 模式，严格按"UI → ViewModel → UseCase → Repository → Data"的分层调用链，禁止跨层访问。



5.2 模块划分

软件代码按职责划分为 8 个顶层包：

```
| 包名 | 职责 | 关键文件 |
| `com.echoling.app` | 应用入口 | `EchoLingApplication.kt` |
| `com.echoling.app.presentation` | UI 层 | `MainActivity.kt`、`viewModel/*`、`ui/*` |
| `com.echoling.app.domain` | 域模型层 | `model/*`、`repository/*`、`usecase/*` |
| `com.echoling.app.data` | 数据层 | `repository/*` (impl)、`local/db/*` |
| `com.echoling.app.player` | 播放引擎 | `AudioPlayer.kt`、`subtitle/*`、`TtsManager.kt` |
| `com.echoling.app.speech` | 语音处理 | `VoskSpeechRecognizer.kt`、`SttRecognizer.kt`、`WordMatcher.kt`、`WavRecorder.kt` |
| `com.echoling.app.di` | 依赖注入 | `DatabaseModule.kt`、`RepositoryModule.kt`、`PlayerModule.kt` |
| `com.echoling.app.utils` | 工具 | (保留扩展位) |
```

5.3 依赖注入 (Hilt)

- **应用入口**：`EchoLingApplication` 标注 `@HiltAndroidApp`
- **Activity**：`MainActivity` 标注 `@AndroidEntryPoint`
- **ViewModel**：标注 `@HiltViewModel`，构造器 `@Inject` 注入 `UseCase`
- **UseCase**：标注 `@Singleton` + `@Inject constructor`，自动注册，无需 Module
- **Repository 接口与实现**：在 `RepositoryModule` 用 `@Binds` 绑定
- **第三方封装**（`AudioPlayer`、`VoskSpeechRecognizer` 等）：标注 `@Singleton` + `@Inject constructor`，自动注册

5.4 协程与并发

- 所有 IO / CPU 密集操作均在 `Dispatchers.IO` 上执行
- ViewModel 通过 `viewModelScope` 管理生命周期内协程
- 状态通过 `StateFlow` / `SharedFlow` 暴露给 UI
- UI 通过 `collectAsState()` 订阅状态，事件回调反向触发 ViewModel 方法

5.5 导航架构

- **Tab 切换**：使用 `HorizontalPager`（4 个主 tab：首页 / 单词本 / 记单词 / 我的），支持左右滑动手势，原生方向感知动画

- ****子页面****：使用 `NavHost` + `NavController`，由 `MainScaffold` 叠加在 Pager 之上（iOS-style overlay）
- ****完整路由****：

```
| 路由 | 入口 |
| `tab_root` | Pager 内默认占位（透明 Box） |
| `course_detail/{courseId}` | 首页课程卡片点击 |
| `practice/{courseId}` | 课程详情页底部按钮 |
| `import` | 首页右下角 FAB |
| `statistics` | 首页统计按钮 / "我的" 入口 |
| `instructions` | "我的" 入口 |
| `permissions` | "我的" 权限说明入口 |
| `category_study/{categoryId}` | "记单词" 分类卡片点击 |
```

6. 模块详细设计

6.1 应用入口模块

6.1.1 `EchoLingApplication`

```
```kotlin
@HiltAndroidApp
class EchoLingApplication : Application() {
 // Hilt 自动生成入口代码；当前无业务逻辑
}
```
```

6.1.2 `MainActivity`

- 单一 Activity 承载整个应用
- `onCreate` 顺序：`setTheme(R.style.Theme\_EchoLing)` → `super.onCreate()` → `enableEdgeToEdge()` → 关闭 `isNavigationBarContrastEnforced`（Android 10+） → `setContent { EchoLingTheme { Surface { MainScaffold() } } }`
- Splash → Main 主题切换在 Activity 创建前通过 manifest `android:theme="@style/Theme.EchoLing.Splash"` 完成

6.2 视图模型层（ViewModel）

| ViewModel | 职责 | 关键 StateFlow |
| `HomeViewModel` | 首页 "继续学习" 卡片 + 进度统计 | `continueLearning: StateFlow<LearningProgress?>` |
| `CoursesViewModel` | 首页 课程列表 | `uiState: StateFlow<CoursesUiState>` (含 `hasLoadedOnce`) |
| `CourseDetailViewModel` | 课程详情 | `course: StateFlow<Course?>` |
| `PracticeViewModel` | 跟读练习 (最复杂 VM: 播放 / 字幕 / 录音 / 进度 / STT 全状态机) | `subtitles`, `playbackState`, `recordingState`, `testState`, `sttTestState`, `wordTranslationState` |
| `VocabularyViewModel` | 单词本列表 | `words: StateFlow<List<Word>>` |
| `ReciteViewModel` | 闪卡分类列表 | `categories`, `progressByCategory` |
| `CategoryStudyViewModel` | 单分类闪卡状态机 | `currentIndex`, `knownCount`, `unknownCount` |
| `StatisticsViewModel` | 学习统计 | `dailyStats`, `monthlyHeatmap` |
| `ImportViewModel` | 导入课程 | `importState: StateFlow<ImportState>` |

6.3 域模型层

6.3.1 数据模型

```
```kotlin
data class Course(
 val courseId: String,
 val title: String,
 val description: String,
 val difficulty: String, // A1/A2/B1/B2/C1/C2
 val audioUri: String?, // SAF Uri 字符串
 val videoUri: String?,
 val subtitleUri: String?,
 val durationMs: Long,
 val totalSentences: Int,
 val thumbnailUri: String?,
 val createdAt: Long,
 val updatedAt: Long
)

data class Sentence(
 val courseId: String,
 val sentenceId: Int,
 val contentEn: String,
```

```

 val contentCn: String,
 val startTimeMs: Long,
 val endTimeMs: Long,
 val isLearned: Boolean = false,
 val isCompleted: Boolean = false,
 val isTested: Boolean = false
)

 data class Word(
 val id: Long = 0,
 val word: String,
 val phonetic: String,
 val pos: String, // part-of-speech
 val translation: String,
 val exampleSentence: String = "",
 val source: String = "manual", // "manual" / "lookup"
 val mastered: Boolean = false,
 val createdAt: Long = System.currentTimeMillis()
)

 data class DictEntry(
 val word: String,
 val phonetic: String,
 val pos: String,
 val translation: String,
 val exampleSentenceEn: String = "",
 val exampleSentenceCn: String = ""
)

 data class DictCategory(
 val id: String, // manifest slug
 val name: String, // 中文分类名
 val description: String,
 val size: Int // 词条数
)
 ...

```

### 6.3.2 仓储接口

5 个核心仓储接口：`CourseRepository` / `SentenceRepository` / `LearningProgressRepository` / `WordRepository` / `DictionaryRepository`。接口定义在 `domain/repository/`，实现在 `data/repository/`，通过 `RepositoryModule.@Binds` 绑定。

### 6.3.3 UseCase 层（共 24 个）

每个 UseCase 封装单一业务逻辑。命名规范：动词+名词（如 `GetCoursesUseCase`、`SaveWordUseCase`）。

主要 UseCase 包括：

- \*\* 课程 相关 \*\* : `GetCoursesUseCase` / `GetCourseDetailUseCase` / `GetCourseSentencesUseCase` / `ImportCourseUseCase` / `DeleteCourseUseCase` / `SyncSentencesUseCase`
- \*\* 进度 相关 \*\* : `GetProgressUseCase` / `SaveProgressUseCase` / `GetContinueLearningUseCase` / `UpdateSentenceCompletedUseCase` / `UpdateSentenceTestedUseCase`
- \*\* 生词本 相关 \*\* : `GetWordsUseCase` / `SaveWordUseCase` / `DeleteWordUseCase`
- \*\* 统计 相关 \*\* : `GetStatisticsUseCase`
- \*\* 词典 相关 \*\* : `LookupWordUseCase` / `GetDictionaryCategoriesUseCase` / `GetDictionaryWordsInCategoryUseCase` / `GetAllDictionaryWordsUseCase`
- \*\* 闪卡 相关 \*\* : `GetReciteProgressUseCase` / `SaveReciteProgressUseCase` / `ObserveAllReciteProgressUseCase`

## 6.4 数据层

### 6.4.1 Room 数据库

参见 § 7 数据库设计。

### 6.4.2 词典数据源

`DictionaryRepositoryImpl` 负责：

1. 首次加载时读 `vocab\_manifest.json`，解析 11 个分类元数据
2. 按需懒加载 `vocab\_<category>.json`，自动嗅探嵌套（4 层）/ 扁平（2 层）两种 schema
3. 用 Gson 将 JSON 反序列化为内部嵌套结构，再 map 为扁平 `DictEntry`
4. 提供 `lookup(word: String): DictEntry?` 单词精确查找
5. 提供 `getCategories() / getWordsInCategory(id) / getAllWords()` 分类与全词查询

### 6.4.3 课程导入流程

1. 用户点击 FAB 触发 SAF `OpenDocument`
2. 用户在系统文件选择器中选取媒体文件

3. ViewModel 拿到 Uri 后异步解析媒体元信息（标题、时长）
4. 查找同目录同名的 `.srt` / `.ass` / `.lrc` 字幕
5. 调用 `ImportCourseUseCase` 写入 `courses` 表
6. 解析字幕文件并通过 `SyncSentencesUseCase` 写入 `sentences` 表
7. 跳转回首页，课程卡片出现

## 6.5 播放引擎模块（Player）

### 6.5.1 `AudioPlayer`

- `@Singleton` 封装 ExoPlayer（Media3 ExoPlayer）
- 对外暴露 `playbackState: StateFlow<PlaybackState>`（含 positionMs / durationMs / isPlaying / isBuffering）
- 提供 `prepare(uri: Uri)` / `play()` / `pause()` / `seekTo(positionMs)` / `setSpeed(speed: Float)` / `release()` 等方法
- 内置 50ms 周期的位置上报循环

### 6.5.2 字幕解析

`SubtitleParserFactory` 根据文件后缀分发到对应 Parser：

格式	Parser	关键点
`.srt`	`SrtParser`	按 `\d+\n` 段落切分，解析 `HH:MM:SS,mmm --> HH:MM:SS,mmm` 时间戳
`.ass`	`AssParser`	解析 `[Events]` 段 `Dialogue:` 行，转换 `H:MM:SS.cs` 为毫秒
`.lrc`	`LrcParser`	解析 `[mm:ss.xx]` 时间戳行；多时间戳合并；处理 ID 标签

解析结果统一为 `List<Subtitle>`，`Subtitle` 包含 `index / startTimeMs / endTimeMs / contentEn / contentCn`。

## 6.6 语音处理模块（Speech）

### 6.6.1 `WavRecorder`

- 封装 Android `MediaRecorder`，输出 16 kHz / 单声道 / 16-bit PCM WAV
- 对外暴露 `recordingState` 和 `amplitude`（50ms 周期）
- 文件落地路径：`context.cacheDir / "rec\_test\_<timestamp>.wav"`

## 6.6.2 `VoskSpeechRecognizer`

- `@Singleton`，封装 Vosk 0.3.45 离线 STT
- **模型来源**：Vosk 离线识别模型（约 68 MB）已**预置**在 APK `assets/models/` 目录中，首次启动时由 `ModelManager.ensureModelReady()` 从 assets 解压至内部存储（一次性 IO，之后复用）
- **完全离线**：模型内置后所有识别操作均在本地完成，无需任何网络请求
- `transcribeFile(wavPath: String): Result<String>`：
  - 校验 WAV 文件头（RIFF/WAVE、44 字节）
  - 创建 `Recognizer(model, 16000f)`
  - **关键修复**：循环调用 `acceptWaveForm` 直至返回 `false`（endpoint detected），每次返回 true 时调用 `recognizer.result` 读取已提交段，最后调 `recognizer.finalResult` 读取尾段，**累加所有段**（旧版只取尾段，会丢失句首段文本）
  - 解析返回 JSON，提取 `text` 字段

## 6.6.3 `WordMatcher`

详见 § 8.2 关键算法。

## 6.6.4 `TtsManager`

- `@Singleton`，封装 Android `TextToSpeech`
- 多引擎 fallback：硬编码 7 个候选 + PackageManager 自动发现的 TTS 服务，按顺序尝试 `setEngine` + `setLanguage(Locale.US)` + `speak`，首个成功的获胜
- 异步 init 期间维护 `pending` 队列，init 完成后 drain
- 防御式：每个 `setEngine` / `setLanguage` / `shutdown` 操作均 try-catch，单引擎异常不会让 app 闪退

## 6.7 UI 层

### 6.7.1 主要屏幕

屏幕	文件	备注
首页（课程列表）	`CoursesScreen.kt`	Hero 卡片 + 课程列表 + FAB
课程详情	`CourseDetailScreen.kt`	课程元信息
跟读练习	`PracticeScreen.kt`	三 Tab 容器
泛听	`ListeningPage.kt`	字幕列表 + 进度条
精听	`SpeakingPage.kt`	单句卡片 + 录音
测试	`TestingPage.kt`	隐藏单词 → 录音 → 对比
单词本	`VocabularyScreen.kt`	生词列表 + TTS

闪卡分类	`ReciteScreen.kt`	11 个分类卡片 + 进度
闪卡学习	`CategoryStudyScreen.kt`	Y 轴翻转卡片 + TTS
统计	`StatisticsScreen.kt`	当日 / 本周 / 月度热力图
导入	`ImportScreen.kt`	媒体选择 + 字幕选择
我的	`MeScreen.kt`	App 信息 + 权限说明 + 联系
启动	`SplashScreen.kt`	1s 显示后切主界面

## 6.7.2 通用组件

组件	文件	用途
`PageHeader`	`components/PageHeader.kt`	48dp 紧凑头部, 替代 M3 TopAppBar
`AppBarBottomBar`	`navigation/AppBottomBar.kt`	自定义 48dp 紧凑底部 Tab Bar
`ContinueLearningCard`	`components/ContinueLearningCard.kt`	Hero 区域"继续学习"卡片
`CourseListItem`	`components/CourseListItem.kt`	课程列表项 (含 4dp accent bar)
`StatsSummaryCard`	`components/StatsSummaryCard.kt`	学习时长 / 单词数 摘要
`CompactConfirmDialog`	`components/CompactConfirmDialog.kt`	240dp 紧凑确认弹窗
`RecordingOverlay`	`practice/components/RecordingOverlay.kt`	录音时的覆盖层
`TestResultCard`	`practice/components/TestResultCard.kt`	测试结果展示 (含逐词命中块)
`TranscriptionEditor`	`practice/components/TranscriptionEditor.kt`	转写文本编辑器

## 7. 数据库设计

### 7.1 数据库概述

- **\*\*引擎\*\***: Room 2.6.x (SQLite 之上)  
 - **\*\*版本\*\***: 5  
 - **\*\*迁移策略\*\***: `fallbackToDestructiveMigration()` (开发期; 正式发布前计划替换为 Migration 策略)  
 - **\*\*单例\*\***: 通过 Hilt 在 `DatabaseModule.@Provides` 中提供 `Room.databaseBuilder(...)`

### 7.2 ER 图 (文字描述)

```
...
courses (1) —————< (N) sentences
courses (1) —————< (N) learning_progress (实际每课程仅 1 条)
vocabulary (独立表, 生词本)
recite_progress (独立表, 闪卡分类进度)
...
```



## 7.3 表结构

### 7.3.1 `courses`（课程表）

列	类型	约束	说明
courseId	TEXT	PRIMARY KEY	UUID 字符串
courseName	TEXT	DEFAULT ""	课程分组名（v3 新增）
title	TEXT	NOT NULL	课程标题
description	TEXT	NOT NULL	课程描述
difficulty	TEXT	NOT NULL	A1/A2/B1/B2/C1/C2
audioUri	TEXT	NULL	音频 SAF Uri
videoUri	TEXT	NULL	视频 SAF Uri
subtitleUri	TEXT	NULL	字幕 SAF Uri
durationMs	INTEGER	NOT NULL	时长（毫秒）
totalSentences	INTEGER	NOT NULL	句子数
thumbnailUri	TEXT	NULL	缩略图 Uri
createdAt	INTEGER	NOT NULL	创建时间（epoch ms）
updatedAt	INTEGER	NOT NULL	更新时间

### 7.3.2 `sentences`（句子表）

列	类型	约束	说明
courseId	TEXT	PK, FK→courses	课程 ID
sentenceId	INTEGER	PK	句子序号（从 0 起）
contentEn	TEXT	NOT NULL	英文
contentCn	TEXT	NOT NULL	中文翻译
startTimeMs	INTEGER	NOT NULL	起始时间
endTimeMs	INTEGER	NOT NULL	结束时间
isLearned	BOOLEAN	DEFAULT 0	是否已学
isRead	BOOLEAN	DEFAULT 0	是否已读
readScore	INTEGER	DEFAULT 0	历史预留字段（当前未使用）
isCompleted	BOOLEAN	DEFAULT 0	是否完成
isTested	BOOLEAN	DEFAULT 0	是否已通过测试

主键：`(courseId, sentenceId)` 联合主键。

### 7.3.3 `learning\_progress`（学习进度表）

列	类型	约束	说明
courseId	TEXT	PRIMARY KEY, FK→courses	课程 ID
currentSentenceIndex	INTEGER	NOT NULL	当前句子

lastPlayedAt	LONG	NOT NULL	上次播放时间	
totalTimeMs	LONG	NOT NULL	累计学习时长	

### 7.3.4 `vocabulary` (生词表)

列	类型	约束	说明	
id	INTEGER	PK AUTOINCREMENT		
word	TEXT	NOT NULL	单词	
phonetic	TEXT	NOT NULL	音标	
pos	TEXT	NOT NULL	词性	
translation	TEXT	NOT NULL	翻译	
exampleSentence	TEXT	DEFAULT ""	例句	
source	TEXT	DEFAULT 'manual'	来源	
mastered	BOOLEAN	DEFAULT 0	是否已掌握	
createdAt	LONG	NOT NULL	添加时间	

### 7.3.5 `recite\_progress` (闪卡分类进度表, v5 新增)

列	类型	约束	说明	
category_id	TEXT	PRIMARY KEY	词库 manifest slug	
current_index	INTEGER	DEFAULT -1	当前卡片 (-1=未开始)	
known_count	INTEGER	DEFAULT 0	认识数	
unknown_count	INTEGER	DEFAULT 0	不认识数	
last_studied_at	LONG	DEFAULT 0	上次学习时间	

## 7.4 DAO 设计

5 个 DAO 接口, 均提供 `Flow<...>` 形式响应式查询:

DAO	关键方法	
`CourseDao`	`observeAll(): Flow<List<CourseEntity>>`、`getById(id): CourseEntity?`、`insert`、`deleteById`	
`SentenceDao`	`observeByCourse(id): Flow<List<SentenceEntity>>`、`updateCompletedStatus`、`updateTestedStatus`	
`LearningProgressDao`	`observeByCourse`、`upsert`、`getLatest: LearningProgressEntity?`	
`WordDao`	`observeAll(): Flow<List<WordEntity>>`、`insert`、`deleteById`、`updateMastered`	
`ReciteProgressDao`	`observeAll(): Flow<List<ReciteProgressEntity>>`、`getByCategory`、`upsert`	

## 7.5 版本历史

版本	变更	迁移方式
v1	→ v2	基础 4 表   基线
v2	→ v3	`courses.courseName` 新列   `MIGRATION_2_3`
v3	→ v4	`vocabulary.pos` 新列   destructive
v4	→ v5	新增 `recite_progress` 表   destructive

## 8. 关键算法设计

### 8.1 跟读测试端到端流程

跟读测试的完整数据流如下：

```
...
用户按住麦克风
 ↓
WavRecorder.start() → 16kHz mono PCM WAV 文件落地
 ↓
用户松开麦克风
 ↓
VoskSpeechRecognizer.transcribeFile(wavPath)
 |—— ModelManager.ensureModelReady() (首次时从 APK assets 解压部署 ~68MB 内置模型到内部存储)
 |—— Recognizer 循环处理 WAV 数据
 | - acceptWaveForm 每次返回 true → 取 recognizer.result 累加段文本
 | - acceptWaveForm 返回 false → 取 recognizer.finalResult 取尾段
 | - 全部段用空格 join → 得到完整识别文本
 |—— 返回 Result<String> (识别文本)
 ↓
WordMatcher.match(originalSubtitle, recognizedText)
 |—— 双向 normalize: 小写 + 去标点 + 按空白 split
 |—— 空文本 → reason="empty_transcription"
 |—— 贪心多集对齐 (详见 § 8.2) → MatchResult(passed, origMatched[], transMatched[], reason)
 ↓
PracticeViewModel 写入 SentenceEntity.isTested = true (仅当 passed=true 时)
 ↓
UI 更新: SttTestState.Transcribed / Passed / Failed
```

↓  
TestingPage 渲染：每词命中块用绿/红背景区分  
...

## 8.2 WordMatcher 词序匹配算法

### 8.2.1 算法目标

将"原字幕英文文本"与"Vosk 转写得到的英文文本"逐词匹配，输出：

- `passed: Boolean` —— 全部原词都有对应转写词视为通过
- `origWords: List<String>` —— 原文本分词结果
- `transWords: List<String>` —— 转写文本分词结果
- `origMatched: BooleanArray` —— 每个原词是否被匹配（与 `origWords` 索引对齐）
- `transMatched: BooleanArray` —— 每个转写词是否被消费（与 `transWords` 索引对齐）
- `reason: String` —— `"ok" / "empty\_transcription" / "wrong\_word"`

### 8.2.2 算法流程

**\*\*步骤 1：标准化\*\***

```
``kotlin
fun normalize(s: String): List<String> =
 s.lowercase()
 .replace(NORMALIZE_REGEX, " ") // 非字母数字的字符替换为空格
 .split(Regex("\\s+"))
 .filter { it.isNotEmpty() }
...

```

将大写转小写，标点替换为空格，按空白拆分为单词列表。

**\*\*步骤 2：空文本短路\*\***

若 `transWords.isEmpty()`，直接返回 `MatchResult(passed=false, reason="empty\_transcription")`，`origMatched` 全 `false`。

**\*\*步骤 3：贪心多集对齐\*\***

```
``kotlin
val origMatched = BooleanArray(orig.size)
val transMatched = BooleanArray(trans.size)

for (i in orig.indices) {

```

```

 for (j in trans.indices) {
 if (!transMatched[j] && fuzzyEqual(orig[i], trans[j])) {
 origMatched[i] = true
 transMatched[j] = true
 break
 }
 }
}

val passed = origMatched.all { it }
...

```

对每个原词，按顺序查找第一个未被消费的、与之"模糊相等"的转写词。找到则双方都标记 **matched**；找不到则该原词未匹配。

**\*\*关键性质：position-independent（位置无关）\*\***——一个原词是否被匹配只取决于"是否有转写词与之等价"，不取决于它们的位置顺序。这避免了"第一个词错了导致后续全错"的级联误判。

**\*\*步骤 4：模糊相等判定\*\***

```

``kotlin
private fun fuzzyEqual(a: String, b: String): Boolean {
 if (a == b) return true
 if (a.length < 2 || b.length < 2) return false
 if ((a to b) in COMMON_ALTERNATES) return true
 if ((b to a) in COMMON_ALTERNATES) return true
 val threshold = when (maxOf(a.length, b.length)) {
 in 0..3 -> 0 // "to" "a" "I" 必须精确匹配
 in 4..5 -> 1 // "were" "have" 允许 1 编辑距离
 else -> 2 // 其它允许 2 编辑距离
 }
 if (threshold == 0) return false
 return levenshtein(a, b) <= threshold
}
...

```

支持 3 类等价：

1. **\*\*完全相等\*\***
2. **\*\*常见交替形态\*\***：`COMMON\_ALTERNATES = {"were", "was"}, {"got", "gotten"}, ...` —— 处理虚拟语气 / 美英差异等语义等价但编辑距离超阈值的情况
3. **\*\*长度缩放 Levenshtein 距离\*\***：短词严格（0 编辑）、中等词 1 编辑、长词 2 编辑

### 8.2.3 Levenshtein 实现

采用迭代两行动态规划,  $O(\text{len}(a) \times \text{len}(b))$  时间、 $O(b)$  空间。STT 转写单词通常短于 20 字符,  $O(n^2)$  完全可接受。

### 8.2.4 测试结果展示

UI 层根据 `MatchResult` 渲染颜色块:

- `origMatched[i] == true` → 原词所在 chip 背景为绿色 (命中)
- `origMatched[i] == false` → 原词所在 chip 背景为红色 (未命中)
- `transMatched[j] == false` → 转写词所在 chip 背景显示为灰色 (多余词)

## 8.3 SRT 字幕解析算法

...

逐行读取文本



按空行 / 连续空行分割为段落



对每个段落:

- └─ 第一行: 解析序号 (整数)
- └─ 第二行: 解析时间戳 "HH:MM:SS,mmm --> HH:MM:SS,mmm"
- └─ 第三行起: 合并为 contentEn
- └─ 若有第四行且以中文字符开头 → contentCn
- └─ 输出 Subtitle(index, startTimeMs, endTimeMs, contentEn, contentCn)

...

时间戳解析: `HH:MM:SS,mmm` → `h\*3600000 + m\*60000 + s\*1000 + mmm`。

## 8.4 闪卡 Y 轴 3D 翻转动画

```
``kotlin
@Composable
fun FlashCard(isFlipped: Boolean, front: @Composable () -> Unit, back: @Composable () -> Unit) {
 val rotation by animateFloatAsState(
 targetValue = if (isFlipped) 180f else 0f,
 animationSpec = tween(durationMillis = 420)
)
 val showBack = rotation > 90f // 90° 时 swap 正反面

 Box(
```

```

 modifier = Modifier
 .graphicsLayer {
 rotationY = rotation
 cameraDistance = 12f * density // 防止 >120° 鱼眼变形
 }
 .aspectRatio(1f) // 强制正方形
){
 if (!showBack) front() else back()
 }
}
...

```

**\*\*关键参数\*\*:**

- `cameraDistance = 12f \* density` —— Compose 默认相机距离太近, >120° 时变鱼眼, × 12 保持 180° 自然透视
- `showBack = rotation > 90f` —— 90° 时卡片边缘对镜头、不可见, 此时交换正反面
- 反面组件预先旋转 180° (`rotationY = 180f`), 外层再旋转后字是正的

## 8.5 11 词库的嵌套 / 扁平 schema 自动嗅探

部分词库 JSON 是 4 层嵌套 (百度翻译导出格式), 部分是 2 层扁平 (自定义格式)。自动嗅探逻辑:

```

````kotlin
fun loadAsset(assetName: String): List<DictEntry> {
    val firstLine = assets.open(assetName).bufferedReader().readLine()
    return if (firstLine.contains("\"wordRank\"")) {
        // 嵌套路径: headWord → content.word.content.trans[]
        parseNested(assets.open(assetName))
    } else {
        // 扁平路径: 直接 List<DictEntry>
        parseFlat(assets.open(assetName))
    }
}
...

```

通过读取首行是否含 `"wordRank"` 字段判定 schema, 避免全文件扫描 (4.6 MB 文件首行 sniff 仅 O(64) 字节)。

9. 界面与交互设计

9.1 设计语言

- **Material Design 3** (M3 1.1.2)
- **主题色**: 深紫系 (§ 11.2 品牌色板)
 - Primary `#7C3AED` (violet-600)
 - OnPrimary `#FFFFFF`
 - PrimaryContainer `#DDD6FE` (violet-200)
 - Secondary `#8B5CF6` / Tertiary `#6D28D9`
- `dynamicColor = false` 强制锁定紫色主题, 不允许被 Android 12+ 壁纸取色覆盖

9.2 关键交互

- | 交互 | 实现 |
- | 4 个主 tab 切换 | `HorizontalPager.animateScrollToPage(target)`, 方向感知 |
- | tab 屏幕左右滑动 | Pager 原生 swipe |
- | 子页面 push | `navController.navigate(...)`, 从右滑入覆盖 Pager |
- | 子页面返回 | `navController.popBackStack()`, 从右滑出, 露出 Pager |
- | 字幕单词点击 / 长按 | `pointerInput { detectTapGestures(...) }` 绕开 M3 ripple |
- | 长按麦克风 | `onPress` + `tryAwaitRelease`, 按住录音, 松停 |
- | 闪卡点击 | `Modifier.clickable { isFlipped = !isFlipped }`, 触发 Y 轴翻转 |
- | TTS 发音 | 点喇叭 `IconButton`, 调 `viewModel.pronounce(word)` |

9.3 状态栏 / 导航栏适配

- `enableEdgeToEdge()` —— 状态栏、导航栏全透明
- 关闭 `isNavigationBarContrastEnforced` (Android 10+) —— 避免系统强制加 scrim 导致 bar 底部发白
- 内层 `Scaffold` 设 `contentWindowInsets = WindowInsets.statusBars` —— 只消费顶部 inset, 让 bottom bar 通过 `Column + weight(1f)` 完美贴底

10. 性能与可靠性设计

10.1 启动性能

- **冷启动** $\leq 1.5\text{ s}$ (实测)
- Splash 显示 1s 期间完成 Hilt 注入、Room 数据库创建、词库 manifest 加载
- 启动期不加载任何词库详细数据, 仅加载 manifest (4 KB)

10.2 内存管理

- 词典 JSON 解析在 `Dispatchers.IO` 上执行，UI 线程零阻塞
- 词库按 category 懒加载，未访问的 category 不解析
- ExoPlayer 单例，所有页面共享一个实例
- Pager 4 个 page 常驻渲染 (offscreen 也保留 composition)，sub-page 单独的 NavHost 实例

10.3 16 KB 页大小对齐 (Android 15+)

Android 15+ 强制要求 native lib 和 zip entry 按 16 KB 对齐。本项目通过三层 hook 实现：

1. ****ELF 内部****：`patchNativeLibsFor16KB` hook 在 `packageDebug.doFirst` 跑，把 `stripped\_native\_libs/` 下每个 `.so` 的 `PT\_LOAD.p\_align` 从 4096 改成 16384
2. ****Zip 外部 (.so)****：`repackApk16kb` hook 在 `packageDebug.doLast` 用 `0xD935` extra block 把每个未压缩 `.so` 的 data offset 推到 16384 倍数
3. ****Zip 外部 (resources.arsc)****：同上，把 STORED entry (含 resources.arsc) 推到 4 字节倍数 (Android R+ 要求)
4. ****重新签名****：用 apksigner v1+v2+v3 重签
5. ****删除 stale .idsig****：AGP 后写的 v4 .idsig 与新签名不一致会引发 `INSTALL\_PARSE\_FAILED\_NO\_CERTIFICATES`，必须删除

10.4 隐私设计

- ****AndroidManifest 不声明 `INTERNET` 权限****——所有网络功能刻意不可用，从根上杜绝任何数据外传可能
- ****Vosk 离线识别模型内置****——约 68 MB 模型直接打包在 APK `assets/models/` 中，首次启动解压至内部存储；运行时所有识别完全本地完成
- ****词典数据打包在 APK assets****——词典查询完全本地
- ****学习数据存本地 Room****——用户进度不上传
- ****SAF 选择媒体****——通过 `OpenDocument` 获取单次读写权限，不需要 `READ\_MEDIA\_\*` 永久权限
- ****「我的」页面提供权限使用说明子页****——在 App 内集中声明每个权限的用途

10.5 错误处理与降级

- | 错误场景 | 降级策略 |
- | Vosk 模型加载失败 | 显示"模型加载失败"卡片，用户可重试或重装 |
- | TTS 引擎全部不可用 | 喇叭按钮可见但点击后显示 snackbar 提示 |
- | 字幕文件不存在 | 导入时弹出提示，用户可手动选择字幕 |
- | 媒体文件损坏 | ExoPlayer 内部错误处理，UI 显示播放错误 |

| 词典查询无结果 | 对话框显示"未找到该单词", 不弹错 |

10.6 并发与生命周期

- 所有协程绑定到 `viewModelScope`, ViewModel 销毁时自动取消
- `SharedFlow` + `extraBufferCapacity` 保护 STT 事件不丢
- Pager 状态用 `rememberPagerState` (内部 `rememberSaveable`), 旋转屏幕后保持当前 tab

11. 测试与验证

11.1 单元测试

- `WordMatcherTest` —— 验证词序匹配算法的关键 case:
 - 完全相同文本 → `passed=true`
 - 单个拼写差异 (Levenshtein 容差内) → `passed=true`
 - 单词遗漏 → `passed=false`, 对应 `origMatched=false`
 - 词序变化 ("If I were you" vs "If I was you") → `passed=true` (COMMON_ALTERNATES 命中)
 - 空转写 → `reason="empty_transcription"`
 - 标点差异 → `passed=true` (normalize 后等价)

11.2 集成验证 (真机实测, 小米 Mi 11 CN)

- 11 个词库全部加载无 crash
- 课程导入: 选 mp4 → 自动找同目录 .srt → 课程卡片出现在首页 → 进详情 → 进跟读练习 → 泛听 / 精听 / 测试三个 Tab 全部正常
- 跟读测试: 按住麦克风说 "If I were you, I would go" → 识别为 "if i was you i would go" → WordMatcher 识别 were=was (COMMON_ALTERNATES) → `passed=true` → 句子标记 `isTested`
- 闪卡: 进入"高中英语词汇" → 翻 5 张卡 → 点不认识 → 进度自动保存 → 杀进程重启 → 仍落在第 6 张
- 长按取词: 长按字幕 "abandon" → 弹"翻译 & 保存"对话框 → 显示音标 / 词性 / 翻译 → 点保存 → 单词出现在单词本

11.3 构建验证

- `./gradlew assembleDebug` BUILD SUCCESSFUL
- APK 通过 `apksigner verify` (v2 + v3 verified)
- `python scripts/repack_apk_16kb.py` 二次执行报告 `(0 → 0 misaligned)`, 证明已对齐
- `outputs/apk/debug/` 下仅有 `app-debug.apk` 与 `output-metadata.json`, 无残留 `.idsig`

11.4 兼容性验证

- Android 8.0 ~ 14+ 真机测试通过
- 浅色 / 深色主题切换正常
- 折叠屏展开 / 折叠后布局自适应
- 横竖屏切换 Pager 状态保持

12. 人员分工与开发周期

12.1 人员构成

> **重要声明**：本软件「听言英语 V1.0」的**全部开发工作由同一人完成**，涵盖需求分析、UI/UX 设计、架构设计、Kotlin 编码、资源准备、构建脚本、测试验证、文档编写的全部环节。

岗位	人数	实际承担工作
产品 / 设计 / 架构师	1	需求分析、UI/UX 视觉规范、架构选型、技术方案
Android 开发工程师	1	Kotlin 编码、Compose UI、Room 数据层、ExoPlayer/Vosk 集成
测试工程师	1	真机功能验证、性能验证、兼容性验证
DevOps / 构建工程师	1	Gradle 配置、16 KB 对齐脚本、签名、构建产物验证
技术文档工程师	1	源代码注释、CLAUDE.md、项目说明文档

> 上述岗位均由**同一名开发者**承担。本节按软件工程规范列出岗位分工，便于审查人员理解工作量分布；实际工作量在 § 12.2 中按时间维度统计。

12.2 开发周期（按工时统计）

阶段	主要任务	工时
需求与设计	需求分析、UI 草图、架构选型、品牌色定义	4 周
应用骨架	项目初始化、Hilt / Compose / Navigation / Room 配置、应用入口、底部导航	1 周
首页与课程模块	首页、课程列表、课程详情、Hero 区域、accent bar	2 周
字幕解析与播放	SRT/ASS/LRC Parser、ExoPlayer 集成、字幕同步	2 周
跟读练习三页	泛听、精听、测试三 Tab + 完整状态机	3 周
STT 与对比算法	Vosk 集成、ModelManager、WordMatcher、词序匹配	3 周
词典与生词本	11 词库加载、嵌套/扁平 schema 嗅探、长按取词、生词本	2 周
闪卡记单词	分类列表、闪卡学习页、Y 轴翻转、进度持久化	2 周
统计与导入	学习统计页面、热力图、SAF 媒体导入	1 周
系统打磨	边缘到边缘、状态栏、底部 bar 贴底、16 KB 对齐	2 周
测试与文档	真机验证、性能调优、CLAUDE.md 编写、软著文档准备	2 周
合计		24 周（约 6 个月）

12.3 关键里程碑

- **M1 (第 2 周) **: App 启动到首页 + 底部导航可切换
- **M2 (第 4 周) **: 导入课程 → 课程详情 → 跟读练习泛听可正常播放
- **M3 (第 8 周) **: 跟读练习三 Tab 完整可用
- **M4 (第 11 周) **: STT 集成完成, 跟读对比测试可用
- **M5 (第 15 周) **: 词典与生词本完整可用
- **M6 (第 17 周) **: 闪卡记单词完整可用
- **M7 (第 22 周) **: 系统打磨 + 16 KB 对齐完成
- **M8 (第 24 周) **: 测试完成, 文档定稿

13. 附录

13.1 关键文件清单

13.1.1 应用入口

- `app/src/main/java/com/echoling/app/EchoLingApplication.kt`
- `app/src/main/java/com/echoling/app/presentation/MainActivity.kt`

13.1.2 核心模块

- `app/src/main/java/com/echoling/app/presentation/ui/navigation/MainScaffold.kt`
- `app/src/main/java/com/echoling/app/presentation/ui/navigation/NavGraph.kt`
- `app/src/main/java/com/echoling/app/presentation/ui/navigation/AppBottomBar.kt`
- `app/src/main/java/com/echoling/app/presentation/ui/navigation/TabPagerHost.kt`

13.1.3 跟读练习核心

- `app/src/main/java/com/echoling/app/presentation/ui/screens/practice/PracticeScreen.kt`
- `app/src/main/java/com/echoling/app/presentation/ui/screens/practice/ListeningPage.kt`
- `app/src/main/java/com/echoling/app/presentation/ui/screens/practice/SpeakingPage.kt`
- `app/src/main/java/com/echoling/app/presentation/ui/screens/practice/TestingPage.kt`
- `app/src/main/java/com/echoling/app/presentation/viewmodel/PracticeViewModel.kt`

13.1.4 语音与匹配

- `app/src/main/java/com/echoling/app/speech/VoskSpeechRecognizer.kt`
- `app/src/main/java/com/echoling/app/speech/SttRecognizer.kt`

- `app/src/main/java/com/echoling/app/speech/WavRecorder.kt`
- `app/src/main/java/com/echoling/app/speech/WordMatcher.kt`
- `app/src/main/java/com/echoling/app/speech/ModelManager.kt`

13.1.5 播放与字幕

- `app/src/main/java/com/echoling/app/player/AudioPlayer.kt`
- `app/src/main/java/com/echoling/app/player/TtsManager.kt`
- `app/src/main/java/com/echoling/app/player/subtitle/SubtitleParser.kt`
- `app/src/main/java/com/echoling/app/player/subtitle/SubtitleParserFactory.kt`
- `app/src/main/java/com/echoling/app/player/subtitle/SrtParser.kt`
- `app/src/main/java/com/echoling/app/player/subtitle/AssParser.kt`
- `app/src/main/java/com/echoling/app/player/subtitle/LrcParser.kt`

13.1.6 数据层

- `app/src/main/java/com/echoling/app/data/local/db/EchoLingDatabase.kt`
- `app/src/main/java/com/echoling/app/data/local/db/entity/CourseEntity.kt`
- `app/src/main/java/com/echoling/app/data/local/db/entity/SentenceEntity.kt`
- `app/src/main/java/com/echoling/app/data/local/db/entity/ReciteProgressEntity.kt`
- `app/src/main/java/com/echoling/app/data/local/db/entity/WordEntity.kt`
- `app/src/main/java/com/echoling/app/data/local/db/entity/LearningProgressEntity.kt`
- `app/src/main/java/com/echoling/app/data/repository/DictionaryRepositoryImpl.kt`

13.1.7 域模型与 UseCase

- `app/src/main/java/com/echoling/app/domain/model/Course.kt`
- `app/src/main/java/com/echoling/app/domain/model/Sentence.kt`
- `app/src/main/java/com/echoling/app/domain/model/Word.kt`
- `app/src/main/java/com/echoling/app/domain/model/DictEntry.kt`
- `app/src/main/java/com/echoling/app/domain/model/DictCategory.kt`
- `app/src/main/java/com/echoling/app/domain/usecase/\*` (24 个)

13.2 资源清单

- `app/src/main/AndroidManifest.xml`
- `app/src/main/res/values/strings.xml`
- `app/src/main/res/values/colors.xml`
- `app/src/main/res/values/themes.xml`
- `app/src/main/res/drawable/ic\_app\_icon.png` (应用图标)
- `app/src/main/res/drawable-nodpi/splash\_image.png` (启动图)

- `app/src/main/assets/vocab\_\*.json` (11 个词库 + manifest)
- `app/src/main/assets/models/` (Vosk 模型)

13.3 构建脚本

- `app/build.gradle.kts`
- `settings.gradle.kts`
- `scripts/gen\_source\_pdf.py` (源程序 PDF 生成)
- `scripts/patch\_native\_libs\_16kb.py` (16 KB 对齐 Layer 1)
- `scripts/repack\_apk\_16kb.py` (16 KB 对齐 Layer 2 + 重新签名)
- `scripts/build\_vocab\_assets.py` (词库资产预处理)

13.4 文档清单

- `CLAUDE.md` —— 项目开发执行手册 (12 大节)
- `听言英语\_V1.0\_程序设计说明书.md` (本文件) —— 程序设计说明
- `听言英语\_V1.0\_源程序\_全本.pdf` —— 源程序全本 (271 页, 113 个 Kotlin 文件)
- `听言英语\_V1.0\_源程序\_前 30 页\_后 30 页.pdf` —— 源程序软著提交版 (60 页)

文档结束

本说明书系统描述了「听言英语 V1.0」Android 应用的整体设计、模块实现、关键算法与人员分工。配套源程序 PDF (`听言英语\_V1.0\_源程序\_前 30 页\_后 30 页.pdf`) 展示了实际代码实现, 可与本说明书交叉印证。

****特别声明**:**

1. 本软件的跟读测试功能采用"语音识别 → 文本 → 与原字幕逐词对比"的方式实现, ****不包含任何打分、评分或发音质量评估机制****。
2. 本软件的全部开发工作由****同一名开发者****独立完成。